ZERO-PREREQUISITE INSTALLATION & RUNTIME GUIDE

Slotbook Fresh PC Setup Guide

One-Command Multi-Container Deployment on Brand-New Windows 10/11

TARGET AUDIENCE:	**REPOSITORY:**
Technical Reviewer / Architect	https://github.com/tiruk007/slotbooking.git
HOST PREREQUISITES:	**TEST VERIFICATION:**
Docker Desktop + Git (Zero Node/SQL needed)	**71 of 71 Tests Passing (100%)**

1. Introduction & Overview

Slotbook is an enterprise-grade appointment scheduling engine built with Next.js 14, TypeScript, and PostgreSQL 16. It guarantees zero double-bookings through kernel-level GiST exclusion constraints (`btree_gist`), handles complex timezone mathematics across Daylight Saving Time (DST) boundaries, and eliminates dual-write failures via the Transactional Outbox pattern.

This document provides a complete, foolproof walkthrough for running Slotbook on a **brand-new Windows 10 or 11 laptop or desktop**. No Node.js or PostgreSQL installations are required on the host system; the entire application and database stack are fully containerized.

—

2. Windows Base Software Installation

For a fresh Windows machine, install the following standard development tools:

2.1 Install Google Chrome (or Edge / Firefox)

Download and install Google Chrome from https://www.google.com/chrome/.

2.2 Install Visual Studio Code

Download and install VS Code from https://code.visualstudio.com/. Recommended extensions:

- **Docker** (`ms-azuretools.vscode-docker`)
- **WSL** (`ms-vscode-remote.remote-wsl`)

2.3 Install Git for Windows

Download and install Git for Windows from https://git-scm.com/download/win. During installation, when prompted under **Adjusting your line ending conversions**, select: `Checkout as-is, commit Unix-style line endings` (recommended to prevent CRLF script corruption).

2.4 Install Docker Desktop for Windows

Download Docker Desktop from https://www.docker.com/products/docker-desktop/. During installation, ensure the checkbox **“Use WSL 2 instead of Hyper-V”** is **CHECKED**. Restart your computer when prompted, then launch Docker Desktop and wait until the whale status icon stays solid green.

—

3. Windows Terminal & WSL 2 Verification

Docker Desktop on Windows utilizes the WSL 2 (Windows Subsystem for Linux) backend for near-native Linux kernel performance:

1. Open **PowerShell** as Administrator (Right-click Start -> **Terminal (Admin)** or **PowerShell (Admin)**).
2. Check WSL status:

```
wsl --status
```

3. If WSL is not installed or outdated, run:

```
wsl --install
wsl --update
```

4. In Docker Desktop Settings (`gear icon` -> `General`), ensure **“Use the WSL 2 based engine”** is active.

—

4. Repository Cloning & Workspace Setup

Open **Git Bash** or **Windows Terminal** (PowerShell):

```
# 1. Navigate to your desired working directory
cd /c/Users/$USER/Downloads

# 2. Clone the repository
git clone https://github.com/tiruk007/slotbooking.git

# 3. Enter the repository
cd slotbooking
```

—

5. Environment Configuration

The repository includes a ready-to-run `.env.example` template with production-grade local defaults.

Create your local `.env` file:

- In Git Bash or WSL:

```
cp .env.example .env
```

- In PowerShell:

```
Copy-Item .env.example .env
```

Key Environment Variables Explained

Variable	Default Value	Description
`PORT`	`3000`	Port on which Next.js web application is exposed.
`DATABASE_URL`	`postgresql://slotbook_user:...@slotbook-db:5432/...`	Points to the containerized PostgreSQL database.
`SESSION_SECRET`	`slotbook_super_secret_session_key_min_32_chars`	Cryptographic encryption key for host session cookies.
`NEXT_PUBLIC_APP_URL`	`http://localhost:3000`	Base URL used for email links and host calendar URLs.
`SMTP_HOST` / `SMTP_PORT`	`smtp.gmail.com` / `587`	Outgoing email host & port (optional for local mock mode).

> **Email Notification Note:**
> For local reviewer evaluation, live SMTP credentials are completely optional. The built-in Transactional Outbox stages notifications into PostgreSQL and logs deliveries cleanly without crashing or failing bookings.

—

6. Single-Command Launch via Docker Compose

In the root of the repository, execute:

```
docker compose up --build
```

Automated Self-Bootstrapping Sequence

1. **PostgreSQL 16 Engine (`slotbook-db`)**:
   - Initializes database cluster and loads the `btree_gist` extension.
   - Mounts a named persistent volume (`postgres_data`) so data survives restarts.
   - Activates health check: `pg_isready -U slotbook_user -d slotbook_db`.
2. **Next.js 14 Application (`slotbook-app`)**:
   - Multi-stage Dockerfile compiles TypeScript and bundles production assets.
   - Executes `docker-entrypoint.sh`:
     - Polls PostgreSQL until healthy.
     - Runs database migrations (`npm run db:migrate`).
     - Runs idempotent seed script (`npm run db:seed`) to create initial host data.
     - Boots the production HTTP server on port 3000.

When the terminal displays:

```
Ready in ...ms
- Local: http://localhost:3000
```

the application is fully operational!

—

7. Step-by-Step Verification & Testing Walkthrough

Follow these steps in Google Chrome or any browser to test all core capabilities:

Test 7.1 — System Health Check

- Open: `http://localhost:3000/api/health`
- Confirms status `200 OK` with database connection verified.

Test 7.2 — Public Customer Booking Experience

1. Navigate to the seeded host booking page: `http://localhost:3000/book/tirumalesh/30-minutes`
2. **Select a Date**: Click any available calendar date.
3. **Timezone Switcher**: Switch between `Asia/Kolkata` (Host's zone) and `America/New_York` (Guest zone). Slot times project dynamically.
4. **Hold Reservation**: Click an available slot (e.g., 9:10 AM). A 5-minute lease is acquired via PostgreSQL GiST exclusion.
5. **Enter Guest Details**: Enter name and email (e.g. `guest@example.com`).
6. **Verify OTP**: Enter the 6-digit OTP (displayed in console or test sandbox `123456`).
7. **Confirmation**: Redirects to confirmation screen with appointment details and reschedule/cancel links.

Test 7.3 — Host Private Portal & Availability Management

1. Navigate to Host Login: `http://localhost:3000/host/login`
2. Enter the seeded host credentials:
   - **Email**: `host@slotbook.internal` (or `tirumaleshkommavarapu@gmail.com`)

- **Password**: `Password123!`

3. **Host Overview**: Inspect confirmed appointments, active event types, and scheduled hours.
4. **Availability Tab**: View weekly recurring rules, add a date override holiday, and click **“Save Weekly Rules”**.
5. **Bookings Tab**: Inspect upcoming, past, and cancelled appointments. Cancel a booking and observe immediate state sync.

Test 7.4 — Run the Automated Test Suite (71 Tests)

Verify that all 71 unit, integration, and concurrency tests pass against the live PostgreSQL container:

```
docker compose exec slotbook-app npm test
```

Expected result:

```
Test Suites: 7 passed, 7 total
Tests:       71 passed, 71 total
Snapshots:   0 total
Ran all test suites.
```

—

8. Common Pitfalls & Windows Troubleshooting

8.1 Port 3000 or 5432 Already in Use

- **Symptom**: `listen EADDRINUSE: address already in use :::3000` or `bind: address already in use 0.0.0.0:5432`.
- **Solution**: Kill the conflicting process in PowerShell:

```
Get-Process -Id (Get-NetTCPConnection -LocalPort 3000).OwningProcess | Stop-Process
Get-Process -Id (Get-NetTCPConnection -LocalPort 5432).OwningProcess | Stop-Process
```

  Or change external ports in `docker-compose.yml` (e.g., `"3001:3000"`).

8.2 Docker Desktop WSL 2 Engine Not Running

- **Symptom**: `error during connect: This error may indicate that the docker daemon is not running.`
- **Solution**: Launch Docker Desktop from Start menu and wait for the whale icon to turn green. If needed, run `wsl --update` in PowerShell (Admin).

8.3 Windows CRLF Line Endings in Shell Scripts

- **Symptom**: `exec /app/docker-entrypoint.sh: no such file or directory` or `\r: command not found`.
- **Solution**: Run `git config core.autocrlf false` followed by `git checkout docker-entrypoint.sh` in Git Bash.

—

9. Daily Development Commands & Teardown

Action	Command
Stop Containers (Preserve Data)	`docker compose stop`
Restart Stopped Containers	`docker compose start`
Tail Live Application Logs	`docker compose logs -f slotbook-app`
Tail Database Logs	`docker compose logs -f slotbook-db`
Run Migrations Manually	`docker compose exec slotbook-app npm run db:migrate`
Re-Seed Database	`docker compose exec slotbook-app npm run db:seed`
Full Teardown (Clean Slate)	`docker compose down -v`

—

Slotbook Production Scheduling Engine

Authored by Tirumalesh Kommavarapu | Verified Repository: github.com/tiruk007/slotbooking